Routing the Silent Half: Per-Unit Choice of Estimator and Evolution Target in Self-Evolving LLM Agents

Abstract

Group-relative policy optimisation assigns zero advantage to any group whose samples score alike, so a unanimous group contributes exactly nothing to the update. We measure this directly: 57.4% of groups in a live run are RL-silent. We show the silent half splits into two regimes that require opposite responses, and that the split is not symmetric — on the solved half a supervised target is *free*, because the group already contains a correct sample; on the unsolved half no target exists unless one is supplied, which makes harness evolution the only consumer that can use those units at all. We therefore make the choice of estimator and the choice of evolution target a per-unit decision rather than a per-run schedule, with the two as orthogonal axes so a single trajectory can feed both. The closest prior work, DAPO's dynamic sampling, acts on exactly this set and responds in the opposite way — it discards the silent groups and oversamples to refill — which on our own measurements costs 1.5–2.4× the rollouts, a multiplier that *grows* with training because the silent fraction itself grows from 0.33 to 0.58 on an arm with no routing at all. We report the negative result that located this leverage: a token-level version of the same idea, reaching 1.7% of tokens, moved held-out accuracy by less than the noise floor. Every capability is flag-gated and reduces to unmodified GRPO when disabled.

1 Method

1.1 Where the gradient is zero

Let a *unit* be a prompt with G sampled answers and binary rewards $r_1, \dots, r_G \in \{0, 1\}$. GRPO centres rewards within the group,

$$A_i \;=\; r_i - \bar{r}, \qquad \bar{r} \;=\; \frac{1}{G}\sum_{j=1}^{G} r_j, \tag{1}$$

so if every sample in the group scores alike then $A_i = 0$ for all i and the unit contributes exactly nothing to the update — all G sequences, every token. Writing p for the per-sample success probability, the probability that a unit carries any signal at all is

$$I_{\mathrm{RL}}(p, G) \;=\; 1 - p^G - (1-p)^G, \tag{2}$$

which is zero exactly at $p = 0$ and $p = 1$ and is maximised at $p = \frac{1}{2}$. Equation (2) is not an approximation: it is the probability that the group is not unanimous, and unanimity is precisely the condition under which (1) vanishes.

Two consequences drive the method.

The silent fraction is large and measurable. Instrumenting a live run (Sec. 3) puts $\hat{P}(\text{silent})$ between 0.29 and 0.57 depending on the training step, at 64 groups per batch: a third to a half of every batch is RL-dead. Even the low end is an order of magnitude above the reach of the token-level shared-prefix channel we measured first (1.7%), and that gap explains, without appealing to noise, why an intervention confined to that channel moved nothing.

The two silent regimes need opposite responses, and they are not equally common. A unit silent because $\hat{p} = 1$ and a unit silent because $\hat{p} = 0$ are both invisible to (1), but they are not the same object, and routing them identically wastes one of them. Measured on the control arm, the silent channel is 87.5% solved (Table 4), which decides which of the two branches carries the method.

1.2 A free target exists on exactly the solved half

The asymmetry is sharper than "different regimes". For a unit with $\hat{p} > 0$, at least one sampled answer was correct, and that answer *is* a supervised target drawn from the model's own rollout. No external teacher is required; this is rejection-sampling fine-tuning. Define

$$\text{selfTarget}(u) \;=\; \mathbb{1}[\hat{p}_u > 0]\,, \qquad \text{target}(u) \;=\; \text{selfTarget}(u) \;\vee\; \text{teacher}(u). \tag{3}$$

The units on which $I_{\mathrm{RL}} = 0$ because $\hat{p} = 1$ are therefore exactly the units on which a target is *guaranteed* to exist at zero marginal cost. The units on which $I_{\mathrm{RL}} = 0$ because $\hat{p} = 0$ are exactly the units on which no target exists unless one is supplied. The gradient-free half of the batch splits cleanly into a half that is free to reuse and a half that is not reusable at all.

1.3 Routing: the harness is a destination, not a schedule

Prior work treats the choice of what to evolve as a property of the *run*: a schedule or a global flag. We make it a property of the *unit*, and we make it two-dimensional. A routing decision is a pair

$$\pi(u) \;=\; \big(m(u),\, h(u)\big), \qquad m \in \{\textsc{rl}, \textsc{sft}, \textsc{skip}\}, \qquad h \in \{\textsc{none}, \textsc{propose}, \textsc{validate}\}, \tag{4}$$

where m names the estimator applied to the unit's tokens and h names what the unit contributes to harness evolution. The two axes are orthogonal by construction, so a single trajectory can feed the gradient and the harness at once. This is deliberate: the two consumers read different things from a trajectory — the estimator reads the tokens, the harness evolver reads the failure mode — and a partition discards one of the readings. Co-Harness's success→model / failure→harness rule is the special case in which π is constrained to be a partition, and we retain it as an ablation (`partition=True`) rather than a rewrite.

The rule implied by (2) and (3) is then forced rather than chosen:

$$\pi(u) \;=\; \begin{cases} (\textsc{sft}_{\text{self}},\ \textsc{validate}) & \hat{p}_u = 1 \quad \text{(RL dead; target free)} \\ (\textsc{sft}_{\text{teacher}},\ \textsc{propose}) & \hat{p}_u = 0 \;\wedge\; \text{teacher}(u) \\ (\textsc{skip},\ \textsc{propose}) & \hat{p}_u = 0 \;\wedge\; \neg\,\text{teacher}(u) \quad \text{(harness is the only consumer)} \\ (\textsc{rl},\ \textsc{none}) & 0 < \hat{p}_u < 1 \quad \text{(RL has signal; leave it alone)} \end{cases} \tag{5}$$

Note the third case. When a unit is unsolved and no teacher is available, the model arm has nothing to learn from it, and any method that only evolves weights must discard the unit. Routing it to the harness is what makes it recoverable at all, which is the concrete sense in which the harness axis is not a convenience.

One trajectory, two consumers. (5) as written still couples the two axes: a unit that RL can learn from contributes nothing to the harness. That is a residual partition, and it discards information — a *mixed* unit contains failed samples, and their failure mode is exactly what a harness evolver reads, while the RL update consumes only the reward. We therefore decouple the harness action with its own threshold τ_h:

$$h(u) \;=\; \begin{cases} \textsc{validate} & \hat{p}_u = 1 \\ \textsc{propose} & \hat{p}_u \le \tau_h \qquad \tau_h \ge \tau_{\text{fail}}, \\ \textsc{none} & \text{otherwise}, \end{cases} \tag{6}$$

so at $\tau_h = \tau_{\text{fail}}$ the rule is unchanged, and raising it lets a unit take $(\textsc{rl}, \textsc{propose})$ — feeding the gradient and the harness from the same trajectory. Under `partition` this is disabled, so the Co-Harness special case is preserved exactly.

Rollback. Every capability above is gated. With the harness arm disabled, $h(u) \equiv \textsc{none}$; with routing disabled, $\pi(u) \equiv (\textsc{rl}, \textsc{none})$ for every unit and the update is bit-identical to unmodified GRPO. Each ablation in Sec. 3 is one flag away from the vanilla baseline, so a difference cannot be attributed to an incidental change of configuration.

1.4 Who decides: from a threshold to a controller

Everything above specifies an action space. What chooses within it is a separate question, and it is where the contribution lies — a rule keyed on $\hat{p}$ is a heuristic, and a learned policy that sees only $\hat{p}$ cannot beat the best threshold on $\hat{p}$ by more than noise. We therefore give the controller features, and instantiate it three ways so the comparison is an ablation rather than a demonstration.

Features, at zero marginal cost. Each unit is described by seven statistics computed from tensors the batch *already carries* — rewards, the loss mask, and the sampled log-probabilities. No extra forward pass, no extra generation. The constraint is deliberate: a feature set needing its own inference budget would have to beat spending that budget on more rollouts, which is a far higher bar than beating a threshold. Two of the seven carry information $\hat{p}$ cannot: the *truncated fraction* separates a unit that failed from one that merely ran out of budget, and the *log-probability dispersion* separates a group unanimous in its answer from one also unanimous in its reasoning. A solve-rate threshold collapses both distinctions by construction.

Three controllers, one action space.

- **Rule** — thresholds on $\hat{p}$, as in (5). The baseline the others must beat, not a strawman: it encodes the exact zero-gradient condition.
- **Learned weights** — LinUCB, one ridge model per mode over the features, updated from observed outcomes. Ridge rather than a network because the outcome channel is confounded and low-signal — one scalar per batch, produced by every decision in it — and capacity would mostly fit that noise.
- **Learned code** — the policy is generated *source*, validated against an allowlist and cost-vetted in a subprocess before adoption. Strictly larger hypothesis class than weights: it can compose features into ratios and nested cases rather than only reweighting them.

Algorithm 1 Per-batch signal routing. $\mathcal{B}$ is a batch of prompts, G samples each.

Require: thresholds τ_{solve}=1, τ_{fail}=0, $\tau_h \geq \tau_{\text{fail}}$; weights $c_+ \geq 0$, $c_- \leq 0$; flags `routing`, `harness`

1: roll out G samples per prompt; score to binary rewards $r_{u,i}$
2: $\hat{p}_u \leftarrow \frac{1}{G}\sum_i r_{u,i}$ ▷ group solve rate
3: $A_{u,i} \leftarrow r_{u,i} - \hat{p}_u$ ▷ group-centred advantages, (1)
4: $\mathcal{S} \leftarrow \{u : \max_i |A_{u,i}| = 0\}$ ▷ RL-silent: exactly the unanimous units
5: **if** `routing` **then**
6: **for all** $u \in \mathcal{S}$ **do**
7: **if** $\hat{p}_u \geq \tau_{\text{solve}}$ **then** ▷ solved: a correct sample is its own target
8: $A_{u,i} \leftarrow A_{u,i} + c_+$ for all response tokens
9: **else if** $\hat{p}_u \leq \tau_{\text{fail}}$ **then** ▷ failed: no self-target exists
10: $A_{u,i} \leftarrow A_{u,i} + c_-$ **if** $c_- \neq 0$, else leave at 0
11: **end if**
12: **end for**
13: **end if**
14: **if** `harness` **then**
15: $\mathcal{P} \leftarrow \{u : \hat{p}_u \leq \tau_h\}$; $\mathcal{V} \leftarrow \{u : \hat{p}_u = 1\}$
16: emit $(\textsc{propose}, u)$ for $u \in \mathcal{P}$ and $(\textsc{validate}, u)$ for $u \in \mathcal{V}$
17: **end if**
18: PPO update on $\{A_{u,i}\}$, clipping unchanged

The control that decides the claim. A fourth arm shuffles which units receive which mode while holding the *proportion* of units in each mode fixed. Without it, a controller that helps cannot be distinguished from a change in how much SFT the run did — and the latter is not a controller at all. Any improvement reported for the learned arms is the improvement over this control, not over the fixed rule alone.

What we verify about the learned arm, and what we do not. On a synthetic environment whose two regions differ *only* in the truncated fraction, with $\hat{p}$ held identical, the contextual controller reaches the optimal mode in both regions across seeds, while a context-free bandit on the same stream provably cannot exceed 0.5 in the worse region. That establishes the mechanism can use a feature a threshold cannot see. It does *not* establish that it learns under the confounded batch-level channel of a real run, which is a separate claim and is not made here.

1.5 The procedure, end to end

Algorithm 1 is the whole method. Every line is either an existing GRPO step or a decision from (5)–(6); nothing else is added, and with routing disabled lines 6–13 do not execute.

Why the update needs no new loss term. Line 8 is the whole of the supervised branch. For a solved unit every $A_{u,i}$ is zero, so adding c_+ makes the objective $c_+ \sum_i \log \pi(y_{u,i})$ over samples already known to be correct — which is supervised fine-tuning on them, with the importance ratio at 1 because the data is on-policy. PPO's clip still bounds the step, which matters because sharpening an already-correct policy is the mechanism's predicted failure. Line 10 is its mirror: $c_- \leq 0$ pushes down samples known to be wrong, and $c_- > 0$ is rejected rather than trusted, since it would train the model to reproduce them.

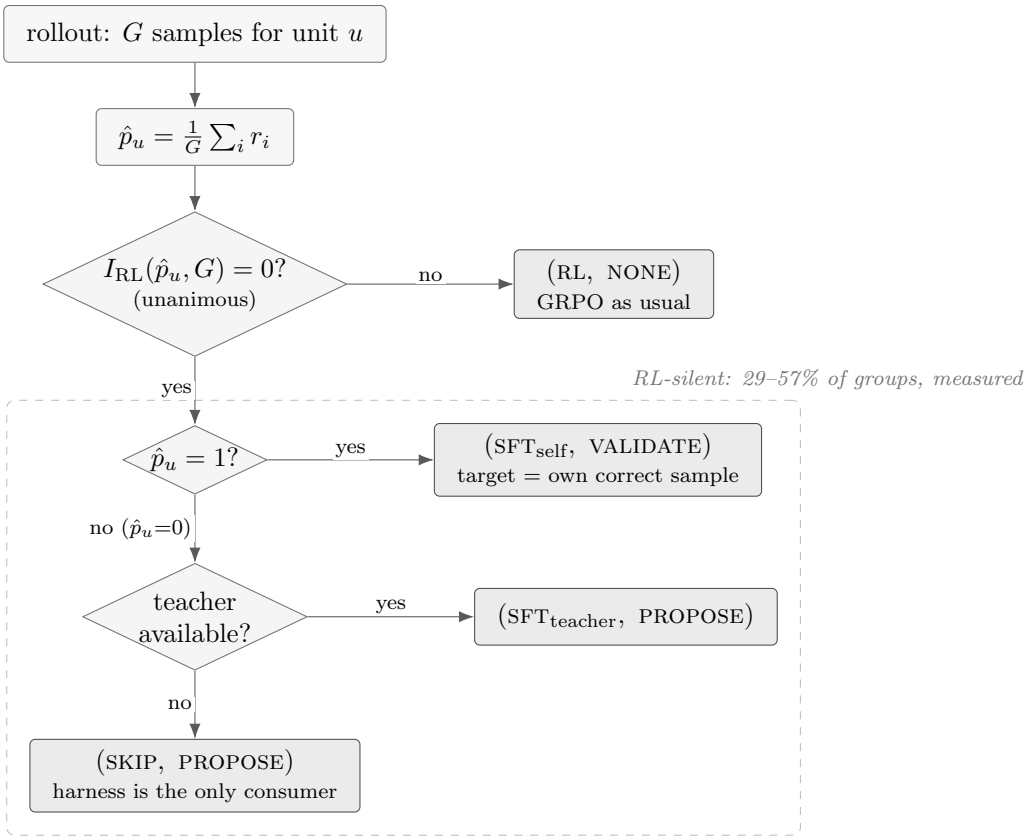

Figure 1: **Routing decides, per unit, both an estimator and a harness contribution.** The branch on I_{RL} is exact, not heuristic: a unanimous group has every advantage identically zero by (1). The dashed region is the part of the batch ordinary GRPO cannot learn from, measured at 29–57% of groups depending on training step. It does not split evenly: 87.5% of it is solved, where a supervised target is free (the unit's own correct sample), and the remainder has no target unless one is supplied — and on that remainder the harness is the only consumer that can use the unit at all.

Cost. Lines 5–13 add no generation. The batch is the batch that GRPO already rolled out; the silent units are re-read, not re-sampled. This is the whole of the difference from DAPO, which reaches the same set by discarding it and paying to generate a replacement.

2 Related work

We organise prior work by the claim it already owns, because our contribution is only meaningful as the complement of those claims.

Discarding the silent groups. DAPO's dynamic sampling is the closest prior work, and it is close in the strongest sense: it acts on exactly the set we act on. Its rule keeps a prompt only when the group's reward standard deviation is positive, and a zero standard deviation is unanimity, which by (1) is precisely the zero-advantage condition. DAPO therefore *detects* the silent channel and *discards* it, oversampling until the batch refills. We reuse it. Same phenomenon, opposite response — so DAPO is the baseline this claim must be measured against rather than a method to cite in passing, and Sec. 3.6 reports what its response costs.

Choosing an algorithm per task. SIA selects among training algorithms at the granularity of a *task*, and demonstrates that the choice matters. That result is theirs and we do not re-derive it. What a task-level policy cannot express is that units *within* one task differ in whether a gradient exists at all: by (2) the silent and informative units of a single task require different estimators, and no task-level assignment can separate them. Our claim is therefore about granularity, and the comparison that matters is against SIA at matched compute.

Deciding what to evolve. Co-Harness co-evolves harness and weights and establishes the success→model / failure→harness assignment. EvoTrainer co-evolves a policy with its training harness. Both fix the assignment as a rule over outcomes. We keep their rule as a special case ((4) constrained to a partition) and relax it, because a trajectory read by two different consumers need not be spent on only one of them.

Deciding *when* to evolve. *Learning, Fast and Slow* separates fast and slow adaptation and reports the stability benefit of a two-timescale schedule. That bounds what a cadence claim can be worth, and we treat cadence as a control variable rather than a contribution.

Clustering trajectories. MEDS reuses layer-wise logits from the forward pass as lightweight representations of reasoning behaviour and clusters error patterns with HDBSCAN. We adopt this directly as the learned instantiation of our cluster key, against a derived key computed from the silence side of (2); the two are a controlled ablation of *how* to cluster, holding what is done with the clusters fixed.

Policies as weights and as code. Contextual bandits are the standard tool for per-instance algorithm selection under a scalar reward, and we use LinUCB unmodified; the contribution is not the estimator but what it is given to see. Generating a policy as executable source is the other established line, and we adopt it as a distinct arm rather than an alternative implementation, because the two differ in hypothesis class rather than in optimiser: weights reweight the features they are handed, code composes them.

Evaluation practice. Recent evaluation work documents how often agent results fail to survive controls that were available at the time. We adopt its checks as requirements rather than as discussion: paired tests on shared items, intervals reported on the *difference* rather than on each arm, matched-permutation controls that hold the proportion of units in each mode fixed while shuffling which units get which mode, and a noise floor established from the same checkpoint scored twice. The matched control is the one that decides our central claim, because a routing method changes both which units are routed and how many, and only the permutation control separates those.

3 Results

All numbers below are measured end-to-end on held-out benchmarks. Train reward is never reported as a result: Sec. 3.1 is the demonstration that it mis-orders checkpoints.

Table 1: MATH-500, $n = 500$, paired McNemar against the scaffold arm. Runs `step0d` (demo, lr 6×10^{-6}, clip 0.4) and `step0h` (scaffold, lr 1×10^{-6}, clip 0.2).

step	demo	scaffold	Δ	McNemar p
base	0.528	0.506	-0.022	same model
28	0.454	0.530	$+0.076$	7.3×10^{-4}
57	0.440	0.530	$+0.090$	1.7×10^{-4}
86	0.466	0.526	$+0.060$	4.1×10^{-3}
115	0.364	0.536	$+0.172$	2.1×10^{-13}
144	0.334	0.522	$+0.188$	9.3×10^{-15}

Table 2: The same checkpoints on OlympiadBench (675 problems), which shares no items with MATH-500.

checkpoint	MATH-500	OlympiadBench
base	0.528	0.188
gs028	0.454	0.170
gs057	0.440	0.190
gs086	0.466	0.148
gs115	0.364	0.135
gs144	0.334	0.108
gs173	0.358	0.107

3.1 Train reward mis-orders checkpoints

Table 1 scores two recipes that differ only in optimiser aggressiveness, on the full MATH-500, paired by item. The aggressive ("demo") recipe's training reward rises while its held-out accuracy falls by 0.194 absolute. Any claim validated on training reward would have selected the collapsed checkpoint.

Noise floor, and what it disqualifies. The base model appears in both series, so its difference is pure measurement noise: 0.0220 at $n = 500$. It *rose* from 0.0080 at $n = 250$ because the larger series was scored in two passes on different server processes, accumulating between-run variance a contiguous sweep does not have. Consequently the $+0.060$ at step 86 is only $2.7\times$ the floor and we do not report it standalone; the $+0.172$ and $+0.188$ are $\approx 8\times$ it. The claim is *preservation*, not improvement: the scaffold arm stays within $[0.506, 0.536]$, which brackets its own base.

3.2 The collapse replicates on an independent benchmark

$0.188 \to 0.107$ is a 43% relative drop in the same direction as MATH-500's $0.528 \to 0.358$, on an independent problem set.

Benchmark selection, stated as a measurement rather than a preference. MATH-500 is saturated at the frontier tier (0.966 at 27B) and AIME is unusable at both ends (0.000 at 1.5B; 0.867 at 27B with a 24-point interval on 30 problems). OlympiadBench resolves a few points on 675 items and leaves both ends off the rails: 0.188 at 1.5B and 0.716 for Ornith-1.5-35B-A3B. The 0.716 is a *lower* bound — 174/675 generations (26%) hit the token cap — so the headroom is smaller than the 28 points it suggests, and we quote it as a bound.

Table 3: MATH-500, $n = 500$. `step0j` adds token-level routing to the scaffold recipe. p is paired McNemar, scaffold vs. scaffold+routing.

step	demo	scaffold	+routing	p
base	0.528	0.506	0.536	0.092
28	0.454	0.530	0.516	0.534
57	0.440	0.530	0.526	0.920
86	0.466	0.526	0.522	0.923
115	0.364	0.536	0.496	0.065
144	0.334	0.522	0.526	0.922

3.3 Negative result: token-level routing at 1.7% reach

Token-level routing changes nothing detectable. We report this rather than dropping it, because it is the measurement that located the method's leverage. The rule reaches at most 1.7% of loss-carrying tokens, and an intervention on 1.7% of the gradient cannot move a benchmark by more than noise; the between-arm noise floor here (0.030 on the shared base checkpoint) is itself larger than every routing difference in the table.

Confound, stated rather than buried. `step0j` differs from `step0h` in three ways — routing, `kl_ctl` ($0.01 \to 0$) and `adv_norm` (batch→off) — because the rule's precondition does not hold otherwise. A significant difference here could not have been attributed to routing alone. The routing-off arm at `step0j`'s exact configuration is therefore a required control, and it is running (`step0l`).

3.4 Where the leverage actually is

Measured live on `step0l` at 64 groups per batch:

$$\widehat{P}(\text{silent}) = 0.574, \qquad n_{\text{groups}} = 64.$$

More than half of every batch contributes exactly zero gradient — 34× the token-level channel, and the reason Sec. 3.3 was null is that the intervention was on the wrong tier, not that the idea was wrong.

A metric bug found and fixed, reported because the affected runs are still cited. The accompanying split first reported 0.000 solved at a measured 82% solve rate, which is impossible. It thresholded `reward_score`, which by that point has had bias, scaling, clipping and normalisation applied. It now reads the raw rewards captured before any transformation. Runs launched before the fix carry the old code and their solved/unsolved split must be ignored; $\widehat{P}(\text{silent})$ is unaffected because it is computed from the advantages being zero.

3.5 The silent channel is 87.5% solved, so most of it needs no teacher

Measured on the routing-off control (`step0l`) under the corrected metric, over 18 logged batches at 64 groups each (1152 group observations):

The identity solved + unsolved = silent holds to a maximum residual of 1.0×10^{-5} across all 18 batches, which is float32 rounding. We check it because an earlier version of this metric failed it, reporting 0.000 solved at a measured 82% solve rate.

Table 4: Decomposition of the RL-silent channel. `step01`, GSM8K, Qwen2.5-1.5B-Instruct, first 18 logged batches.

quantity	mean	range
silent groups	0.3592	[0.2891, 0.4414]
silent because *solved*	0.3145	[0.2461, 0.4219]
silent because *unsolved*	0.0447	[0.0117, 0.0703]
solved share of the silent channel	**0.875**	[0.827, 0.959]

Table 5: Rollout cost of discarding versus reusing the silent channel. s is measured on the routing-off arm; the multiplier is $1/(1-s)$.

point in run	silent fraction s	DAPO	ours
first 10 batches	0.327	1.49×	1.00×
whole run	0.525	2.10×	1.00×
last 10 batches	0.583	2.40×	1.00×
max observed	0.633	2.72×	1.00×

This inverts the cost of the method. By (3), the solved groups already contain a supervised target, so 31.4% of *all* groups are reusable with no teacher, no extra inference and no extra GPU. Only 4.5% of groups — the unsolved ones — require an external teacher, and those are exactly the units for which the harness is the only available consumer. The reach of the free path is therefore about 7× that of the teacher-dependent one, which is the opposite of the ordering we assumed before measuring.

Scope, and what this does not claim. This is one operating point: GSM8K, Qwen2.5-1.5B-Instruct, early training, high solve rate. A harder task moves mass from solved to unsolved and shifts the balance toward needing a teacher; 0.875 is a property of the operating point, not a constant. More importantly, the table says where the reachable mass *is*, not that training on it helps. Sharpening an already-correct distribution spends entropy, and entropy collapse is the failure mode Sec. 3.1 already documents. Until the A/B exists, the solved branch of (5) defaults to SKIP.

3.6 What discarding the silent channel costs

DAPO's response to the same channel is to drop it and regenerate. If a fraction s of groups is unanimous, refilling a batch requires generating $1/(1-s)$ groups per accepted group. Substituting the silent fractions measured on the control arm (98 batches) gives Table 5.

The gap widens with training. The silent fraction is not a constant: it rises from 0.33 to 0.58 within 98 batches. Crucially this was measured on the arm with *no routing at all*, so the growth is a property of RL training rather than of our intervention — as the policy improves, more groups become unanimous and GRPO's effective batch shrinks. An overhead that grows as the model gets better is the wrong shape for the regime frontier models occupy.

Stated as a prediction, not a result. $1/(1-s)$ is the expectation under the assumption that regenerated groups are unanimous at the same rate. The DAPO arm measures the true multiplier from the accept/reject counts rather than inheriting this estimate. We fix the comparison axis in

Table 6: The silent channel does not shrink on a harder task — it inverts. Qwen2.5-1.5B-Instruct in both columns.

	GSM8K	MATH
silent groups	0.359	0.419
silent because solved	0.315	0.164
silent because unsolved	0.045	**0.255**
solved share of the channel	**87.5%**	**39.1%**
unsolved share	12.5%	**60.9%**

advance as *matched generation budget*, not matched steps: DAPO's kept batch is denser per step, so an equal-step comparison would flatter it.

3.7 The solved branch is inert, and then harmful

Acting on the free self-target does not help. At $c_+ = 0.5$ the effect is null at every checkpoint and the difference-in-differences after removing the base offset is 0.000–0.002 at four of five steps. At $c_+ = 2.0$ it becomes negative: -0.054 at step 144 ($p = 0.014$, 2.5× the noise floor). We report it as suggestive rather than conclusive — one hit in five paired tests does not survive a Bonferroni threshold of 0.01 — but the *shape* argues it is real: negative or zero at four of five steps with the largest effect at the *last* checkpoint, which is what cumulative over-sharpening looks like and is not what a random hit looks like. It agrees with an independently measured entropy trace in which the routed arm descends faster mid-run.

The mechanism is unflattering and simple: a group is solved *because* the policy already places high probability on the correct answers, so supervising on them teaches what the model already knows.

3.8 The composition of the silent channel inverts with task difficulty

If that were the whole story the channel would be worthless, because on GSM8K it is 87.5% solved. It is not the whole story. Holding the model and configuration fixed and changing only the training task:

The unsolved fraction of *all* groups rises from 4.5% to 25.5%, a 5.7× increase, while the solved fraction roughly halves. So the branch we measured to be worthless shrinks to a minority exactly where the task gets hard, and the branch for which no self-target exists — and for which a teacher or the harness is the only possible consumer — becomes the majority.

This is what the routing rule of (5) is for. A fixed threshold tuned on GSM8K would spend its effort on the solved side, which is where the effort is wasted; keying on the *side* of silence rather than on a tuned constant is what survives the shift. We state the limits plainly: $n = 11$ batches for the MATH column against 98 for GSM8K, one model at one scale, and — the lesson of Sec. 3.7 — locating mass is not the same as showing that acting on it helps.

3.9 The learned router does not beat its matched control

Sec. 1.4 names the control that decides this claim: `router=random` run at the contextual arm's *measured* mode proportions, so the two arms differ only in *which* unit receives which mode and not in the mixture. Those proportions were read off the last 40 steps of the contextual run (`ctx`, 64 groups per step) — rl 0.2946, sft 0.3527, skip 0.3526 — and the control (`rnd`) was run at exactly them. Both

Table 7: Learned router (`ctx`) against its matched-proportion control (`rnd`), both at `globalstep149`, greedy. Wilson 95% intervals in brackets. MATH-500, AMC23 and AIME are scored at `max_tokens`=3072 and OlympiadBench at 16384; arms are compared only against each other at the same budget.

benchmark	n	ctx	rnd	ctx − rnd
MATH-500	500	0.5240 [0.480, 0.567]	0.5260 [0.482, 0.569]	−0.0020
OlympiadBench	675	0.1837 [0.156, 0.215]	0.1837 [0.156, 0.215]	+0.0000
AMC23	40	0.1750 [0.087, 0.320]	0.1250 [0.055, 0.261]	+0.0500
AIME24	30	0.0000 [0.000, 0.114]	0.0000 [0.000, 0.114]	0.0000
AIME25	30	0.0000 [0.000, 0.114]	0.0333 [0.006, 0.167]	−0.0333

arms are scored from `globalstep149` of their own run, GSM8K training, Qwen2.5-1.5B-Instruct, greedy decoding (temperature 0, seed 0). That step is the matched comparison point rather than a chosen one: the contextual run stalled at step 162 of 290 while the control reached 245, so 149 is the latest checkpoint the two arms share.

Stated positively rather than as an absence: **at matched mode proportions, the per-unit routing decision buys nothing over assigning the same modes at random.** What this routing arm sets is the *mixture*; which unit receives which mode carries no effect the suite can measure.

Which rows can carry that claim, and which cannot. Only the first two. MATH-500's −0.0020 is one tenth of the 0.020 systematic noise floor measured for greedy scoring on this suite (Sec. 3.1 measures 0.0220 on MATH-500 itself), and OlympiadBench returns the same score on both arms to four decimals over 675 problems. The other three rows resolve nothing: at $n = 40$ and $n = 30$ the Wilson intervals span more than 20 points, so AMC23's +0.0500 and AIME25's −0.0333 — one problem out of thirty — sit well inside them, and AIME24's 0.000 on both arms is this scale's known floor rather than a comparison. One caveat on the intervals themselves: they are per-arm Wilson intervals, not the paired intervals on differences this section reports elsewhere. The paired test was not run for this pair, so what bounds the two resolving rows is the noise floor, not an interval on the difference.

What a real effect on OlympiadBench would have to clear. A single greedy score at $n = 675$ carries about a point of jitter: the *same* `ctx` checkpoint on the *same* 675 problems scored 0.1941 at `max_tokens`=8192 and 0.1837 at 16384, with nothing changed but the budget. A future arm difference below roughly two points on this benchmark is therefore not evidence, and +0.0000 is far inside that. Both arms also leave generations that never emit a boxed final answer at any budget — 78/675 for ctx and 64/675 for rnd at 16384, against 79 and 61 at 8192 — so the truncation these rows carry is non-termination rather than a budget-bound score, and doubling the cap does not remove it.

Scope, and what this does not claim. One seed, one checkpoint, one training task, one model scale. It falsifies "this router, with this credit signal, beats its matched control". It does not show that routing cannot help, and by construction it says nothing about the mixture, on which the two arms are matched. Sec. 3.10 localises the failure, and the failure is not the bandit.

Table 8: Mode mix under per-batch credit. `ctx`, GSM8K, Qwen2.5-1.5B-Instruct, first 129 logged steps at 64 groups per step. L_1 is the distance of the quarter's mean mix from uniform thirds; 1.33 would be total concentration on one mode.

quarter (steps)	rl	sft	skip	L_1 vs. uniform
Q1 (1–32)	0.305	0.341	0.354	0.056
Q2 (33–64)	0.299	0.338	0.363	0.069
Q3 (65–96)	0.351	0.303	0.346	0.061
Q4 (97–129)	0.344	0.320	0.336	**0.027**

3.10 Why the router does not learn: the credit signal, not the bandit

This subsection and the next report the controller's own training-time behaviour — the mode mix it emits — and not capability. They are diagnostics of a mechanism, not results on a held-out benchmark, and are labelled as such throughout.

Under per-batch credit the router never developed a preference. Table 8 gives its mode mix by quarter over the first 129 logged steps of `ctx`.

The mix stays at uniform thirds and the trend is flat to *decreasing* — the last quarter is the most uniform of the four. A learner should move *away* from uniform as its arm estimates separate; this does the opposite.

Feedback was flowing, not blocked. The absence is not a plumbing failure. Over those 129 steps the router received 128 feedback records, each covering all 64 units of its batch, with exactly one batch in the whole run refused as confounded (a batch whose units all took one mode carries no comparison, and its update is skipped rather than made). The batches were not degenerate either: the most common mode's share stays between 0.352 and 0.766; all three modes are present in 95 of the 128 records, and the count never falls below 2.25; and the weak-attribution flag, which fires when one mode exceeds 90% of the units, reads 0 in 108 of the 128 and never exceeds 0.5. Both counts are averaged over four data-parallel ranks, which is why they are fractional: a value below 3 means one rank's view of the batch was short a mode, not that the batch itself was. The router got 128 updates on batches that carried a comparison and formed no preference from them.

The mechanism, as an argument. LinUCB accumulates, for the arm m that was chosen,

$$A_m \leftarrow A_m + xx^\top, \qquad b_m \leftarrow b_m + r\,x, \qquad \theta_m = A_m^{-1} b_m.$$

Per-batch credit gives every unit in the batch the same r, and under a cold start that assigns modes independently of the context x, an arm chosen n_m times accumulates $A_m \approx n_m\,\mathbb{E}[xx^\top]$ and $b_m \approx r\,n_m\,\bar{x}$, so

$$\theta_m \approx \left(n_m\,\mathbb{E}[xx^\top]\right)^{-1} r\,n_m\,\bar{x} = \mathbb{E}[xx^\top]^{-1} r\,\bar{x},$$

which is *the same vector for every arm*: the per-arm count cancels. The arms start identical and stay near-identical, selection is left to the exploration bonus, and the mix stays uniform. This rules out "the router needs more steps": an uninformative signal does not become informative with repetition.

The mechanism, as a measurement. The argument is turned into a test in `selfevo/tests/test_credit_assig` which drives the same registry-built router for 60 batches of 64 units and changes *only* the credit (seed 0, the default the test runs at). Under a shared per-batch scalar the largest pairwise distance

Table 9: Mode mix under per-prompt credit. `ctxpc`, GSM8K, Qwen2.5-1.5B-Instruct, first 85 logged steps at 64 groups per step; the run was still training when these were read. Compare L_1 against 0.027–0.069 for per-batch credit (Table 8).

quarter (steps)	rl	sft	skip	L_1 vs. uniform
Q1 (1–21)	0.905	0.046	0.049	**1.144**
Q2 (22–42)	0.381	0.405	0.214	0.239
Q3 (43–63)	0.000	0.353	0.647	0.667
Q4 (64–85)	0.006	0.431	0.562	0.655

between two arms' parameter vectors is 0.124 and no mode takes more than 0.379 of the units (the test asserts < 0.5 and < 0.55). Under credit that depends on which mode was applied, the same router separates to 1.500 and the rewarded mode takes 0.989 of the units (asserted > 1.0 and > 0.5). Same bandit, same contexts, same number of updates, opposite outcome — so the null of Sec. 3.9 localises to the credit signal, not to the controller or its exploration.

3.11 Per-prompt credit changes the router's behaviour; centring the credit does not

The implied fix is to credit each decision with its own prompt's change in solve rate between the batch where it was routed and that prompt's next appearance, rather than with one scalar per batch. Table 9 is the same measurement as Table 8 on the arm that does this (`ctxpc`, identical to `ctx` except for the credit).

The mix stops being uniform, and it revises: L_1 swings between 0.239 and 1.144 where per-batch credit never left 0.027–0.069. Only the credit changed — same router, same features, same exploration — so the difference is attributable to the signal.

The transition is abrupt, and it coincides with the first credit. Per-prompt credit reaches nothing until a prompt recurs. `prompt_credit/observed_units` is exactly 0 for steps 1–28 and first non-zero at step 29 (4.25 of 64 units); the first step whose `rl_groups` is 0 is step 30, the step immediately after. Before the credit arrives the arm is not routing on evidence at all: after three cold-start steps near uniform thirds (0.352, 0.328, 0.332 rl), steps 4–29 sit at exactly 1.000 rl. That is the documented tie-break artifact — with no `observe` calls the arm parameters never move, the scores tie, and `argmax` resolves alphabetically to RL — so the pre-transition plateau is not a preference either.

Centring the credit changes essentially nothing, which kills the obvious confound. The credited value is a prompt's change in solve rate across roughly 29 steps of training, over which the policy improves generally; the signal therefore conflates "the model got better" with "this mode helped this prompt". Subtracting the batch's mean delta removes the common component. Matched over the first 89 logged steps of each arm:

The two arms are the same arm to within a third of a point in every window, and the common trend that centring removes is small: `prompt_credit/mean_delta` is exactly 0 for the 28 steps before credit flows and lies in 0.005–0.090 (median 0.049) afterwards. Removing the common trend does not move the behaviour, so **the training-progress confound is not the explanation** for the collapse away from RL.

Table 10: RL share of the batch under raw (`ctxpc`) and batch-mean-centred (`ctxpcc`) per-prompt credit, matched over the first 89 logged steps of each arm.

steps	raw	centred
1–15	0.867	0.870
16–30	0.933	0.933
31–89	0.003	0.001

The surviving hypothesis, labelled as one. What remains is that the transition is driven by the *exploration bonus* rather than by the credit values: the heavily-used arm is the first to have its A updated, so its confidence bonus is the first to shrink, and the untouched arms become relatively more attractive whatever the credit says. This predicts exactly the value-independence observed above, and it predicts self-correction once the bonuses equilibrate. The second prediction is at least not contradicted — the raw arm's rl share returns from 0.000 across steps 43–63 to a mean of 0.336 over steps 120–132 (0.26 at step 130) and 0.367 over steps 115–152, the last step logged when this was read — but that is an observation on a run that is still training, not a test. **This has not been tested.** Two things would settle it. Logging the two terms of the LinUCB score separately at the transition: bonus exhaustion requires that the point estimates $x^\top \theta_m$ are still tied when the ranking flips, and is dead if they had already separated in favour of SFT and SKIP before step 30. And re-running the same credit with the exploration coefficient at zero: if the transition survives, it was never the bonus.

Scope, and what this does not claim. One arm per credit variant, one seed, one task, one scale, and both runs unfinished at the step counts quoted. Nothing here is a capability claim: **neither credit arm has been scored on the held-out suite**, and a mix that moves is not a mix that helps — especially this one, which drives `rl_groups` to zero, so that no group receives an ordinary policy gradient and every group is either supervised on its own correct sample or skipped. Sec. 3.7 measures that branch inert at $c_+ = 0.5$ and harmful at $c_+ = 2.0$, which is a reason to expect this regime to cost capability rather than to gain it. Until a scored arm exists, this is a demonstration that the credit signal controls the router's behaviour, and nothing more.

3.12 A benchmark score is a property of the token budget it was taken at

The scoring harness resolved generation parameters by applying a per-benchmark defaults table unconditionally, so an explicit `--max-tokens` was parsed, accepted, and then discarded. A run made with the wrong budget still succeeds and still writes a plausible number, so the defect is invisible from the score alone. Table 11 re-scores one 30B checkpoint (`Frontis-MA1-30B`, a `Qwen3MoeForCausalLM` mixture of experts) after fixing the precedence: same model, same server, same concurrency, same grader, and the only thing that differs between the two halves of each block is which cap reached the sampler.

AIME24 moves by a factor of three and AIME25 by a factor of 2.6, from a flag that appeared on the command line of every earlier score and never reached the sampler.

Both corrected rows are still lower bounds. 20% of generations reach even the 32768 cap and are graded wrong, so 0.8000 and 0.7000 bound this checkpoint from below rather than measuring it. OlympiadBench carries a second and unrelated lower-bound caveat at every cap: 94 of its 675

Table 11: One 30B checkpoint scored at the cap that was requested and at the cap that was applied. `cap asked` is what the command line requested — the same value in every row — and `cap used` is what the sampler received. Nothing else changed.

benchmark	n	cap asked	cap used	accuracy	truncated
AIME24	30	32768	8192	0.2667	73.3%
AIME24	30	32768	32768	**0.8000**	20.0%
AIME25	30	32768	8192	0.2667	73.3%
AIME25	30	32768	32768	**0.7000**	20.0%

Table 12: Length-knee onset across six GSM8K runs on Qwen2.5-1.5B-Instruct. c_+ is the constant written onto solved groups. "steps" is the number of logged training steps available for the run, so the control rows record how long a crossing had to fail to appear.

run	group routing	c_+	steps	onset
`ctx`	on	0.5	162	144
`ctxpc`	on	0.5	152	132
`sa2`	on	2.0	145	131
`step0l`	off	—	273	none
`step0j`	off	—	178	none
`g16`	off	—	116	none

problems state more than one gold answer inside a single answer string, which the exact-match grader marks wrong however the model answers.

What made the defect visible, and it was not the score. The corrected code prints the budget it resolved (`NOTE ... BENCH_OVERRIDES default 8192 not applied`), and the results row records the generation parameters the run actually used, so the requested cap can be compared against the applied one. Recording the parameters a run used is not bookkeeping; without it a cap defect produces no symptom at all, because the run still completes and the number it writes is still in range.

The general form, which outlives this particular bug. A default that silently outranks an explicit request is indistinguishable, from outside, from the request being honoured. We therefore state the convention rather than assume it: **a benchmark number is a property of the token budget unless the budget is reported**, and a comparison taken across two budgets is not a comparison.

3.13 Length moves like a threshold, and routing *presence* is what separates

Mean response length during training does not drift upward under routing; it holds a plateau and then breaks out of it. We fix the onset criterion before reading the runs: onset is the first step of five consecutive steps whose mean response length exceeds the run's own pre-onset baseline by more than four of that baseline's standard deviations.

All three runs with group routing configured hold a flat plateau for more than 120 steps and then cross; none of the three runs without it ever crosses, over 273, 178 and 116 steps. Slope brackets the change: 0.34–0.57 tokens per step before onset across the three routed runs, and 5.6–9.2 after.

The same separation appears in free generation. Scored outside training, greedily, from the checkpoint ladders: `ctx` grows 327 → 455 tokens, while `step0l` moves 330 → 316 and is flat over 202 steps.

Presence, not magnitude. We state the surviving claim in the narrowest form the data supports. Whether group routing is *configured* predicts whether a run crosses; the *magnitude* of the constant does not predict when it crosses, which is the negative result of Sec. 3.14. Three runs on each side is enough to notice a clean split and not enough to estimate an effect, and the six runs are not a matched sweep: `g16` trains at $G = 16$ where the others use $G = 8$, `step0j` carries the token-level rule of Sec. 3.3 and is unrouted only at the group level, and the runs were allowed different numbers of steps.

How each run was classified, because the obvious way is wrong. Routing status is read from `actor.group_routing` in each run's `config.yaml`, never from the presence of `route/*` metrics in its log. `sa2` emits no `route/*` metric at all while its config carries `group_routing.enabled: true` with `solved_advantage: 2.0` and `router: null` — the actor's fixed-rule branch writes the constant without passing through the metric seam. Classified from metrics alone, `sa2` would have been counted on the wrong side of Table 12, and it is the run carrying the larger constant.

3.14 Three negative results on the routed arms

(a) The learned router shows no preference over its matched random control. Sec. 3.9 is that result and we do not restate its table: at the contextual arm's own measured mode proportions the per-unit decision is worth −0.0020 on MATH-500 and +0.0000 on OlympiadBench, against a 0.020 noise floor, and Sec. 3.10 localises the cause to the credit signal rather than to the controller.

(b) The dose-response between constant magnitude and knee onset is not supported. We had recorded — and described as the strongest evidence available — that the larger the constant written onto solved groups, the earlier the length knee arrives. Under the onset criterion of Sec. 3.13 it does not hold. $c_+ = 0.5$ gives onsets at steps 132 (`ctxpc`) and 144 (`ctx`); $c_+ = 2.0$ gives 131 (`sa2`), which sits inside the spread of the two 0.5 arms rather than ahead of them. Three runs at two operating points cannot resolve a dose-response, and the earlier claim is withdrawn here rather than quietly not repeated.

(c) The runs that genuinely produced non-terminating output were the *unrouted* ones. Reading training-time `no_eos_ratios` across all 18 training runs logged for this study, the routed arm `ctx` peaks at 0.0059 — six of 1024 rollouts. The two runs that produced non-terminating output at any material rate are `step0b` (0.156) and `step0` (0.117), and **both have group routing off**. Both use the same aggressive hyperparameters as the demo recipe of Sec. 3.1 (lr 6×10^{-6}, clip 0.4), which that section already measures to destroy capability.

The caveat that keeps (c) from being stronger than it is. `no_eos_ratios` is mis-instrumented: it compares sequence lengths against the *padded* batch width, so it can only count rows that reach that width and it under-counts. Every value it reports is a lower bound. That is enough to establish the positive half of (c) — `step0b` and `step0` really did produce non-terminating output — and not enough on its own to establish the negative half. What supports the negative half is Sec. 3.15, which measures the routed arm's stop probability directly, and the routed arm's free greedy output, which grows to 455 tokens against a 1024 training cap rather than running away from it.

Table 13: Integrated EOS hazard over 64 frozen greedy responses, teacher-forced through the routed ladder (`ctx`) and the group-routing-off control (`step01`). Four of the 27 probed checkpoints, at steps both ladders share.

	`ctx` (routed)			`step01` (control)	
step	H	p_{end}	H_{body}	H	H_{body}
24	0.99754	0.99754	$< 5 \times 10^{-7}$	0.99924	$< 5 \times 10^{-7}$
74	0.99958	0.99958	$< 5 \times 10^{-7}$	1.00204	0.00208
99	0.99975	0.99976	$< 5 \times 10^{-7}$	**1.20534**	**0.20538**
149	0.99958	0.99958	$< 5 \times 10^{-7}$	1.09105	0.09110

3.15 Teacher-forced EOS hazard: the routed arm's termination signal does not erode

Sec. 3.13 leaves two candidate mechanisms that the training logs cannot separate, and both are statements about the probability of stopping. Under *smooth erosion*, an objective that raises the log-probability of tokens that were produced gradually lowers the probability the model assigns to the stop token. Under a *dynamical threshold*, that probability holds and then collapses. We measured the quantity rather than argued about it.

Protocol. One set of 64 greedy responses is generated once and frozen. Each of 27 checkpoints — the routed ladder (`ctx`), the matched control with group routing off (`step01`), and the base model as step 0 — is then teacher-forced over those same token sequences, and we read the model's probability of the stop token at every position. Writing T for the position of the true final token,

$$H = \sum_{t \le T} p_{\text{eos}}(t), \qquad p_{\text{end}} = p_{\text{eos}}(T), \qquad H_{\text{body}} = H - p_{\text{end}},$$

so H is the integrated stop hazard over a response, p_{end} is the spike at its semantic end, and H_{body} is stop-mass placed anywhere else.

Two things pinned before the numbers mean anything. The stop token was determined empirically rather than assumed: all 64 frozen responses end on 151645 (`<|im_end|>`) and none on 151643. Alignment between the scored positions and the text was checked by the strongest test available here — the frozen responses are greedy, so a correctly aligned teacher-forced pass must reproduce them by argmax. Agreement at the generating checkpoint is 0.998, with $p_{\text{end}} = 0.99928$; an off-by-one gives approximately zero on both.

The routed arm's hazard does not erode. Across the ladder it rises, from 0.99754 at the earliest probed checkpoint to a maximum of 0.99979, and the minimum over the 64 prompts rises with it, $0.850 \to 0.994$. H_{body} stays below 5×10^{-7} at every probed step, so the hazard is a spike at the response's semantic end and that spike does not move. We quote the range rather than a trend because the four steps in Table 13 are not themselves ordered (0.99975 at step 99 against 0.99958 at 149, a difference of 1.7×10^{-4}); what the ladder supports is that H stays within 2.5×10^{-3} of 1 throughout and at no point declines toward zero.

This refutes both mechanisms we had entertained. Smooth erosion predicts a falling H, and it does not occur. The dynamical-threshold account is refuted at its premise rather than at its prediction: the hazard is not the failing state variable at all, because it never leaves a 2.5×10^{-3}

band while the same arm's free greedy output grows from 327 to 455 tokens. Whatever the length increase is, it is not the model losing its ability to stop.

Not a stale-text artifact. The responses are frozen, so a sufficiently drifted checkpoint could in principle be scored on text it would never itself produce. A self-generated control repeats the measurement with each checkpoint scored on its *own* greedy output ($n = 32$): `ctx`'s mean p_{end} stays in $[0.984, 0.9999]$ while its own greedy length grows $324 \rightarrow 469$.

The control is not flat, and we had claimed it was. `step01`'s H rises to 1.205 at step 99. Its end spike is pinned at 0.99996 throughout, so the extra 0.205 of stop-mass sits *mid-response*, on 16 of the 64 responses, reaching 1.81 on one. The control drifts toward stopping *earlier*, which is the opposite direction from the routed arm's length growth and therefore does not confound the comparison — but the claim that it was flat was wrong, and is corrected here rather than dropped.

What this retires on the fix side. The codebase offers exactly two explicit termination terms, `overlong_reward_penalty` and `mask_no_eos_with_zero`, and both are disabled in every arm reported in this paper. Either one acts on p_{eos}, which is already 0.9996 at the true end and does not move. There is nothing there for it to correct, so if the routed constant needs to be made safe the lever is the constant itself, not a termination penalty added beside it.

Limitations of the termination and length measurements. Four, stated together because each bounds Secs. 3.13–3.15. (i) *The post-knee regime is unmeasured.* `ctx`'s last saved checkpoint is step 149; the run continued to 162 with mean length still climbing (647 and rising) and was then killed abruptly rather than stopped on a criterion, so a hazard collapse living past the last checkpoint is not something these probes could see. (ii) *The probes are greedy and the training was not.* The frozen and self-generated sets are both greedy, while training sampled at temperature 1.0. Greedy length at step 149 (455) tracks the training-time sequence length at step 145 (478), so greedy is a fair proxy for the mean, but the sampled tail is not measured. (iii) $n = 64$ prompts for the frozen probe and 32 for the self-generated control: the hazard differences we are calling null are small, and so is the sample they are null over. (iv) *The control is itself moving*, as the paragraph above records, so Table 13 compares a flat routed arm against a drifting control rather than against a fixed reference.

3.16 Frontier comparison at an honest cap

Sec. 3.12 establishes that a benchmark number without its budget is not a measurement. Table 14 applies that convention to the two frontier candidates for the fixed strong base: `Qwen3.8-27B` and `Frontis-MA1-30B`, both scored on the same box with the same grader at an explicit 32768 cap, with the cap-precedence fix of Sec. 3.12 confirmed live on that box by the line the corrected code prints, `NOTE aime24: using explicit --max-tokens=32768` followed by `(BENCH_OVERRIDES default 8192 not applied)`. Truncation is reported beside every accuracy, because at this cap it is the column that says whether the accuracy beside it is a measurement or a lower bound.

The truncation column carries as much of the comparison as the accuracy column. `Qwen3.8-27B` is 0.8% cap-limited on MATH-500 against `Frontis-MA1-30B`'s 11.4%, so its MATH-500 figure is close to a measurement while every `Frontis-MA1-30B` figure in the table remains a lower bound. The smaller model wins on all five benchmarks and is less cap-limited on all five,

Table 14: Frontier comparison at a matched, explicit 32768 cap, greedy, one run per model. `trunc` is the number of generations that reached the cap and were therefore graded wrong. AMC23 is included for completeness and is read as saturated rather than as a score.

		`Qwen3.8-27B`		`Frontis-MA1-30B`	
benchmark	n	accuracy	trunc	accuracy	trunc
MATH-500	500	**0.9760**	4/500 (0.8%)	0.8980	57/500 (11.4%)
AMC23	40	1.0000	0/40	0.9500	2/40
AIME24	30	**0.9000**	4/30 (13.3%)	0.8000	6/30 (20.0%)
AIME25	30	**0.9000**	3/30 (10.0%)	0.7000	6/30 (20.0%)
OlympiadBench	675	**0.7733**	120/675 (17.8%)	0.7363	135/675 (20.0%)

which is why the paper's fixed strong base is `Qwen3.8-27B`: a delta measured on it is both harder to obtain and harder to dismiss as scale.

AMC23 at 1.0000 **is saturation, not a score.** With $n = 40$ and zero errors the Wilson interval is $[0.912, 1.000]$, so the benchmark cannot separate two models that both sit here and the row resolves nothing. We report it and then decline to use it: AMC23 has no resolving power left at this capability and is excluded from any comparison that has to separate strong models.

A matched cap is not automatically a fair cap. Both models truncate at 32768, so neither number measures capability; both measure *capability under a* 32768 *budget*, which is a legitimate quantity but not the one a bare ranking implies. Where a cap binds on both sides, part of the gap is verbosity rather than skill, and the ranking is trustworthy only if it is stable as the cap grows. On OlympiadBench `Frontis-MA1-30B` is visibly still on the slope — 0.6237 at 16384 (33.3% truncated), 0.7363 at 32768 (20.0%), with 65536 not run — so that row in particular is a comparison of two lower bounds. Sec. 3.17 supplies the stability check for the other four.

Scope. One run per model, greedy, one set of caps. AIME at $n = 30$ carries about 0.1 of run-to-run jitter on this pipeline, so 0.9000 against 0.8000 on AIME24 is about one jitter width and is not significant on its own; the MATH-500 gap, 0.9760 against 0.8980 at $n = 500$, is well outside it and is the row that carries the ranking. Both OlympiadBench figures inherit the grader caveat of Sec. 3.12: 94 of the 675 problems state more than one gold answer in a single answer string and are marked wrong however the model answers.

3.17 The 32768 cap is saturated: doubling the budget buys essentially nothing

The fairness objection to Table 14 is that a cap which binds on both sides partly measures verbosity. The check is to raise it. Table 15 re-scores `Qwen3.8-27B` at 65536: same model, same grader, only the cap changed.

MATH-500 moves +0.004 at $n = 500$, where the standard error is about 0.006 — inside noise. Each AIME set moves by exactly one problem out of thirty, well inside the ≈ 0.1 jitter recorded at that n. The informative quantity is the pair: **truncation more than halves while accuracy does not follow**, which is the signature of a budget that was already sufficient. The generations being cut off at 32768 were ones that were going to be graded wrong regardless. For MATH-500 and both AIME sets, therefore, the 32768 numbers of Table 14 are measurements rather than lower bounds, and the ranking there can be stated plainly.

Table 15: `Qwen3.8-27B` at 32768 against the same model at 65536. Truncation more than halves on every benchmark while accuracy barely moves. OlympiadBench at 65536 is still running and is reported as not yet measured rather than omitted.

benchmark	@32768	trunc	@65536	trunc	Δ
MATH-500	0.9760	0.8%	**0.9800**	0.2%	+0.004
AIME24	0.9000	13.3%	**0.9333**	6.7%	+0.033 (1 problem)
AIME25	0.9000	10.0%	**0.9333**	3.3%	+0.033 (1 problem)
OlympiadBench	0.7733	17.8%	not yet measured		—

Table 16: An accidental repeat run. Four checkpoints from the `ctx2` (contextual) and `rnd` (random-router control) ladders, each scored twice on MATH-500 at a matched 16384 cap. *spread* is $|A - B|$.

checkpoint	run A	run B	spread	truncation
`rnd`@173	0.4600	0.4580	**0.002**	10%
`ctx2`@173	0.4580	0.4480	**0.010**	11%
`ctx2`@199	0.2660	0.2420	**0.024**	96%
`rnd`@199	0.3480	0.3220	**0.026**	99%

OlympiadBench is the one that is still open. It is also the one where truncation at 32768 was high enough for verbosity to matter (17.8% against `Frontis-MA1-30B`'s 20.0%). Its 65536 measurement is re-running; the first attempt at that cap is void for the reason given in Sec. 3.20 and is not quoted here or anywhere else in this paper.

A caveat that belongs with these numbers. The 65536 run used concurrency 40 against the earlier run's 24, so the batch composition differed between the two columns. Decoding is greedy and the requests are independent, so no answer depends on which others ran beside it; but batched matmuls do not associate identically across batch sizes, and a one-problem AIME difference is exactly the resolution at which that could contribute. This does not weaken the conclusion, which rests on the accuracy *not* moving while truncation halved, and not on the sign of +0.033.

3.18 The noise floor is a function of truncation

`narrow.sh` was launched twice by mistake, so four checkpoints were each scored twice on MATH-500 at a matched 16384 cap. The duplication measures the run-to-run spread of this exact pipeline directly, rather than by assertion, and it does so at two very different truncation rates.

At $\approx 10\%$ truncated the same checkpoint reproduces to 0.002–0.010; at $\approx 97\%$ truncated it moves by 0.024–0.026, an order of magnitude worse. The noise floor is therefore not one number for this suite — it is a function of how much of the batch the cap is cutting off.

The rule this licenses. Above roughly 80% truncation an accuracy is dominated by which few generations happened to finish, and finishing is not random: the survivors are the fastest generations, hence the shortest. **We do not compare accuracies taken above $\approx 80\%$ truncation**, and where two such points differ we read the truncation channel instead. This had until now been argued rather than measured; Table 16 is the measurement.

What it retires. We had recorded that the random-router arm *retains more accuracy* than the contextual one after the transition of Sec. 3.19 — at step 289, 0.3260 against 0.1740. Both points sit

Table 17: MATH-500 truncation at a matched 16384 cap across both ladders. Accuracies are deliberately not tabulated past the transition: by Sec. 3.18 they are not comparable there, and truncation is the trustworthy channel.

step	149	173	174	199	202	231	260	289
`ctx2` (contextual)	6.4%	11%	13.2%	**96%**	99.4%	92.2%	77.8%	96.4%
`rnd` (random)	6.0%	10%	12.4%	**99%**	99.6%	99.6%	97.4%	98.2%

at 96–98% truncation, where the repeat spread is 0.025, so what that comparison reports is which few generations finished on each arm rather than a capability difference. The sign survives; the magnitude does not, and neither does any ordering among the post-transition points. We withdraw the reading here rather than quietly not repeating it.

3.19 The collapse transition narrows to steps 174–199, and both arms cross together

Every point in Table 17 is MATH-500 truncation at a matched 16384 cap, on the checkpoint ladders of the contextual arm (`ctx2`) and the random-router control (`rnd`). The cap is 16384 and not 32768 because these are 1.5B checkpoints with `max_position_embeddings: 32768`; asking such a model for 32768 *new* tokens is rejected outright (Sec. 3.20). It is 5.3× the 3072 these arms were originally scored at.

The unmeasured window is twenty-five steps wide, and both arms cross inside it. Truncation goes 13.2 → 96 on the contextual arm and 12.4 → 99 on the random one between step 174 and step 199. The two ladders agree to within a percentage point at every measured step on either side of that window — 12.4 against 13.2 before, 99.6 against 99.4 after.

This is the strongest form of the controller-is-not-the-cause result. The random router has no learned policy, no credit signal and no preference over modes (Sec. 3.9), so the timing of this transition cannot be a property of the controller. It is a property of the routed constant, which both arms share. The earlier evidence for that conclusion was that both arms collapse; this is the stronger statement that they collapse *at the same step*.

It is also not a cap artifact, which was a live possibility. The original step-289 scores were taken at 3072 with ≈ 100% truncation, where an accuracy is a truncation rate wearing accuracy's clothing. Raising the budget 5.3× moves truncation from ≈ 100% only to 96–98%. These policies genuinely emit more than 16384 tokens; the pathology is in the model and not in the budget.

Two wrong shapes from one missing interval, recorded because the error repeats. This table has now corrected the reading of the same transition twice, in opposite directions. From training-time length curves on runs killed at step 162 we first called the change *threshold-like*; from four points at 149/174/260/289 we then corrected that to *gradual, not a cliff*; filling in 202, 231, 173 and 199 shows it is *sharp*. Both wrong shapes came from drawing a line through the same unmeasured gap. The general form is worth more than the particular correction: interpolating a shape across an interval that contains no measurement produces a confident claim whose sign is set by which endpoints happened to exist.

What the table does not support. There is no clean post-collapse plateau. After the transition the contextual arm reads 99.4 → 92.2 → 77.8 → 96.4, non-monotone across four points; its step-249 checkpoint (not shown, outside the matched step set) is the oddest, at 61.4% truncation with the ladder's lowest accuracy, so at that checkpoint the failures are not all length. That is unexplained and we do not model it.

3.20 Three ways a harness reported a number that was not a measurement

Sec. 3.12 reports one defect of this kind. It was not the only one, and the three together have a shape worth stating as a contribution rather than as an apology: in all three the run completed, exited zero, and wrote a plausible number, so nothing downstream of the score could have caught them.

(i) A defaults table silently outranking an explicit cap. `resolve_params` applied a per-benchmark `BENCH_OVERRIDES` table unconditionally, so an explicit `--max-tokens` was parsed, accepted and discarded. AIME24 read 0.2667 at an applied 8192 against 0.8000 at the requested 32768, and AIME25 0.2667 against 0.7000: a factor of three from a flag that appeared on every command line and reached no sampler. Table 11 is that measurement.

(ii) A cap larger than the model's context, which fails every request. A re-score asked 1.5B checkpoints with `max_position_embeddings: 32768` for 32768 *new* tokens. The server rejected every request, and the scorer reported `acc=nan` with `fail=500/500`. A `nan` in an accuracy column reads as a score to any reader who does not read the failure column beside it, which is the entire argument for printing that column.

(iii) A client timeout discarding work the server had already done. Concurrency was raised 16 → 40 on a 65536 OlympiadBench run while the launcher held a fixed `--timeout 600`. The KV pool was never the constraint; more sequences sharing the cards means each takes longer, and per-request latency crossed the fixed deadline. The server's access log records 613 generations returned with `200 OK`; the client kept 13 of 675 and counted 662 failures. The scorer then reported `acc=0.6154, n=13/675, fail=662` and exited zero. Roughly a GPU-day of generation produced thirteen usable answers, and the 0.6154 is void: it is an average over self-selected survivors, which are the fastest generations, hence the shortest, hence not a random sample of the benchmark.

Why a warning is not a guard. The scorer did print that its accuracy was over survivors and biased upward. That is correct and it is not enough, because a warning beside a plausible number is read as a number. The abort-on-failure-rate guard that would have stopped this at 98% failures had been written, tested and committed — to a repository the scoring box cannot reach, so the box was still running a copy transferred by hand hours earlier. A fix that exists only in version control does not protect a machine that cannot reach version control. The progress monitor compounded it: it counted server-side completions, which is all a server log can show, and read 90.8% complete for a run that yielded thirteen answers.

The convention we adopt, stated as a requirement on reporting. Every benchmark number in this paper is reported with three things beside it: **the token budget it was taken at, the fraction of generations that hit that budget, and the number of requests that failed**. Any one of the three defects above is invisible without one of them, and each produced a number in

the plausible range. The generalisation is that a harness which cannot fail cannot measure: if a run has no path that aborts, then a successful exit carries no information, and the score it wrote is a property of the harness's defaults rather than of the model.

3.21 Not yet measured

Stated explicitly so the gap is not mistaken for an omission: whether training on the solved branch helps or costs entropy (Sec. 3.5 locates the mass but does not settle this); any arm in which routed units *gain* a signal rather than only losing one; and the harness-destination arm of (4). Until a target is supplied, every routed arm is a gradient-deletion ablation and is labelled as one.

Reproducibility notes

Every number in Sec. 3 names the run that produced it. Benchmarks are scored with the same grader across arms; the grader's own bias was measured and corrected once (an earlier version discarded unbalanced final boxes, which penalised degraded checkpoints specifically). Intervals are reported on differences, paired by item, never as independent per-arm intervals.